

Lab 5. US Baby Names

The **United States Social Security Administration (SSA)** provides data on the frequency of baby names in the U.S. from **1880 to the present**. During this project, we will analyze trends in baby names. The first step is to **download the dataset**.

Data source: <https://www.ssa.gov/oact/babynames/limits.html>

Task 1. (Loading the data)

Download the archive: **National data (7Mb)**. Read the **NationalReadMe.pdf** file to understand the data structure. Note that there is a separate CSV file for each year.

Load all yearly files into **one DataFrame**:

1. Define the range of years `years` (from 1880 to 2024):
2. Define the column names for each file (`name` , `gender` , `births`).
3. Create an empty list that will store DataFrames for each year: `data_frames = []` .
4. Loop over each year `for year in years :`
 - Define the file path, e.g., `names/yob' + str(year) + '.txt'`
 - Load the data to a DataFrame `df` using `pd.read_csv()` .
 - Assign column names.
 - Add a new column `year` with the current year.
 - Append the DataFrame to the list `data_frames.append(df)` .
5. Concatenate all DataFrames into one a DataFrame `names` using `pd.concat()` (see `pd.concat`).

```
In [2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [59]: years = [x+1880 for x in range(2024-1880+1)]
columns = ["name", "gender", "births"]
data_frames = []

for year in years:
    path=f"names/yob{year}.txt"
    data=pd.read_csv(path, names=columns)
    data["year"]=year
    data_frames.append(data)

names=pd.concat(data_frames)
names.head()
```

```
Out[59]:
```

	name	gender	births	year
0	Mary	F	7065	1880
1	Anna	F	2604	1880
2	Emma	F	2003	1880
3	Elizabeth	F	1939	1880
4	Minnie	F	1746	1880

Task 2. (Total number of births per year)

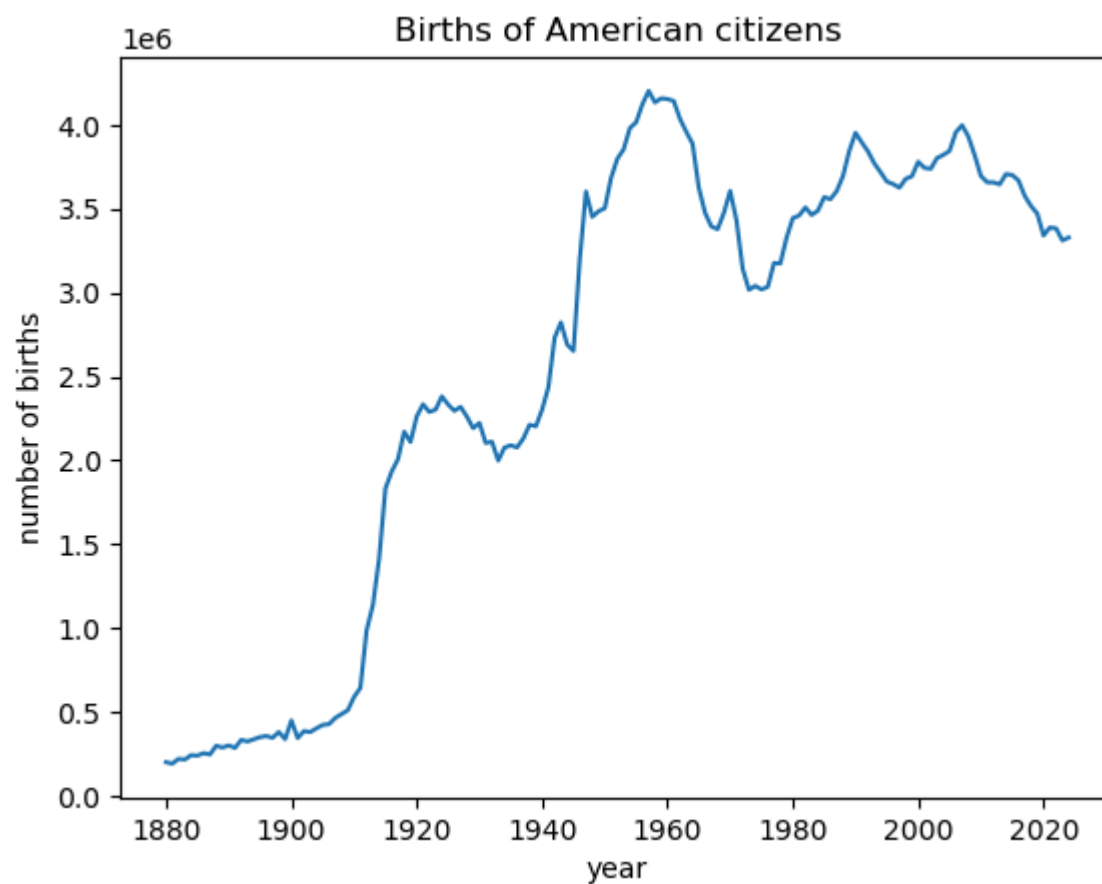
Let us analyze the **total number of births per year**.

1. Group the data by the column `year` .
2. Sum the values of the `births` column for each year.
3. Store the result in a new DataFrame (or Series).
4. Plot the results to visualize the trend (include `title` , `xlabel` , `ylabel`).

```
In [30]: gr_names=names.groupby(names["year"])
birth_sum=gr_names["births"].sum() #type(birth_sum)==Series
plt.plot(birth_sum)
plt.title("Births of American citizens")
plt.xlabel("year")
plt.ylabel("number of births")
```

```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x753fdc376850>
```

```
Out[30]: Text(0, 0.5, 'number of births')
```



Task 3. (Total number of births per year by gender)

Now, we want to see how the number of births has changed **separately for male and female names**.

We will build a **pivot table** where:

- the **index** is the `year`,
- the **columns** are the genders (`F` and `M`),
- the **values** are the total number of births.

Finally, plot the results to visualize the trend (include `title`, `xlabel`, `ylabel` and `legend`).

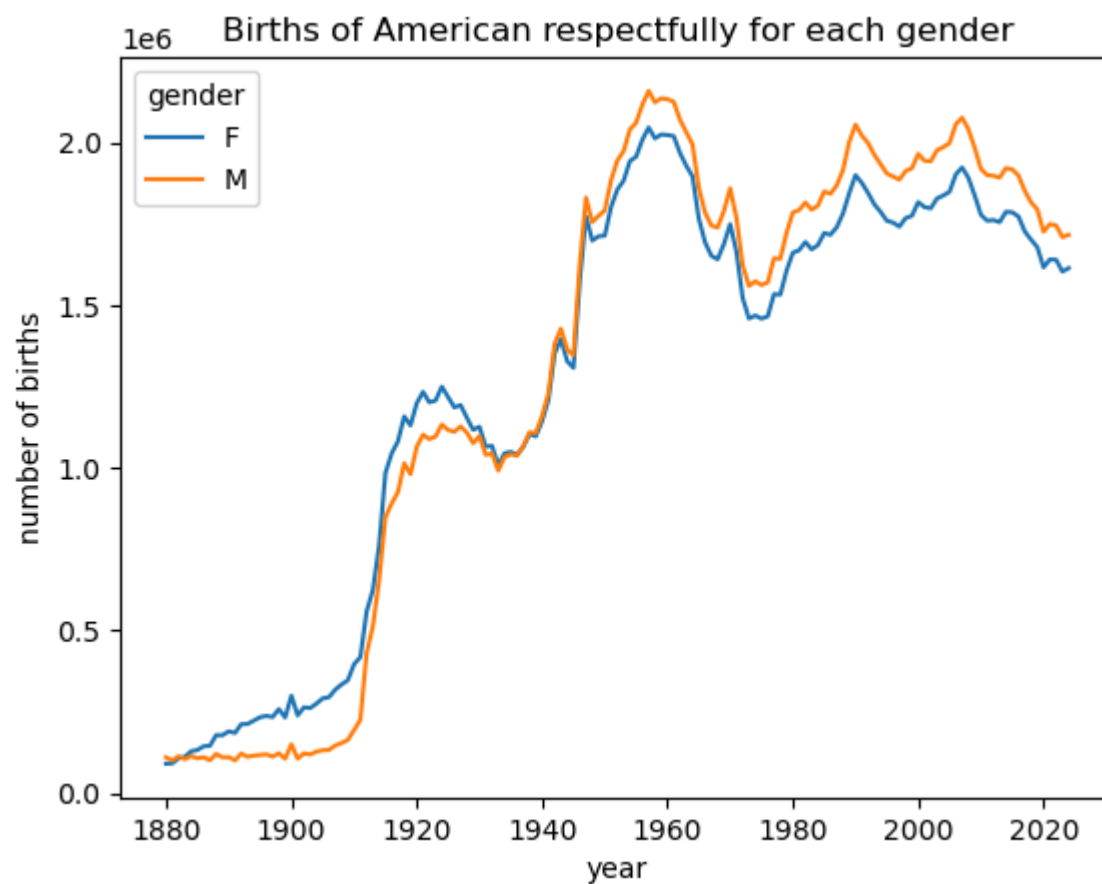
```
In [5]: birth_sum_gen=names.pivot_table("births",
    index="year",
    columns="gender",
    aggfunc="sum")
```

```
)  
print(birth_sum_gen)  
birth_sum_gen.plot()  
plt.title("Births of American respectfully for each gender")  
plt.xlabel("year")  
plt.ylabel("number of births")
```

gender	F	M
year		
1880	90994	110490
1881	91953	100735
1882	107847	113686
1883	112321	104625
1884	129019	114442
...	...	...
2020	1615512	1725065
2021	1639712	1748237
2022	1638762	1744373
2023	1602645	1708551
2024	1613188	1715313

[145 rows x 2 columns]

Out[5]: Text(0, 0.5, 'number of births')



Task 4. (Number of different names over the years)

In this task, we want to analyze how the **diversity of names** has changed over time, **separately for female and male names**. Specifically, we will count how many **unique names** appear in each year.

1. Group the data by `year` and `gender`.
2. For each group, count the number of **unique names**.
3. Store the result in a new DataFrame.
4. Pivot the data so that years are rows and genders are columns.
5. Plot the trend (include `title`, `xlabel`, `ylabel` and `legend`) to compare diversity between female and male names.

In [156...

```
gr_2=names.groupby(["year", "gender"])
unique_names=pd.DataFrame(gr_2["name"].count()) #points 2. and 3.
piv=unique_names.pivot_table(index=["year"],columns="gender")
print(f"Pivoted data:\n{piv}")
```

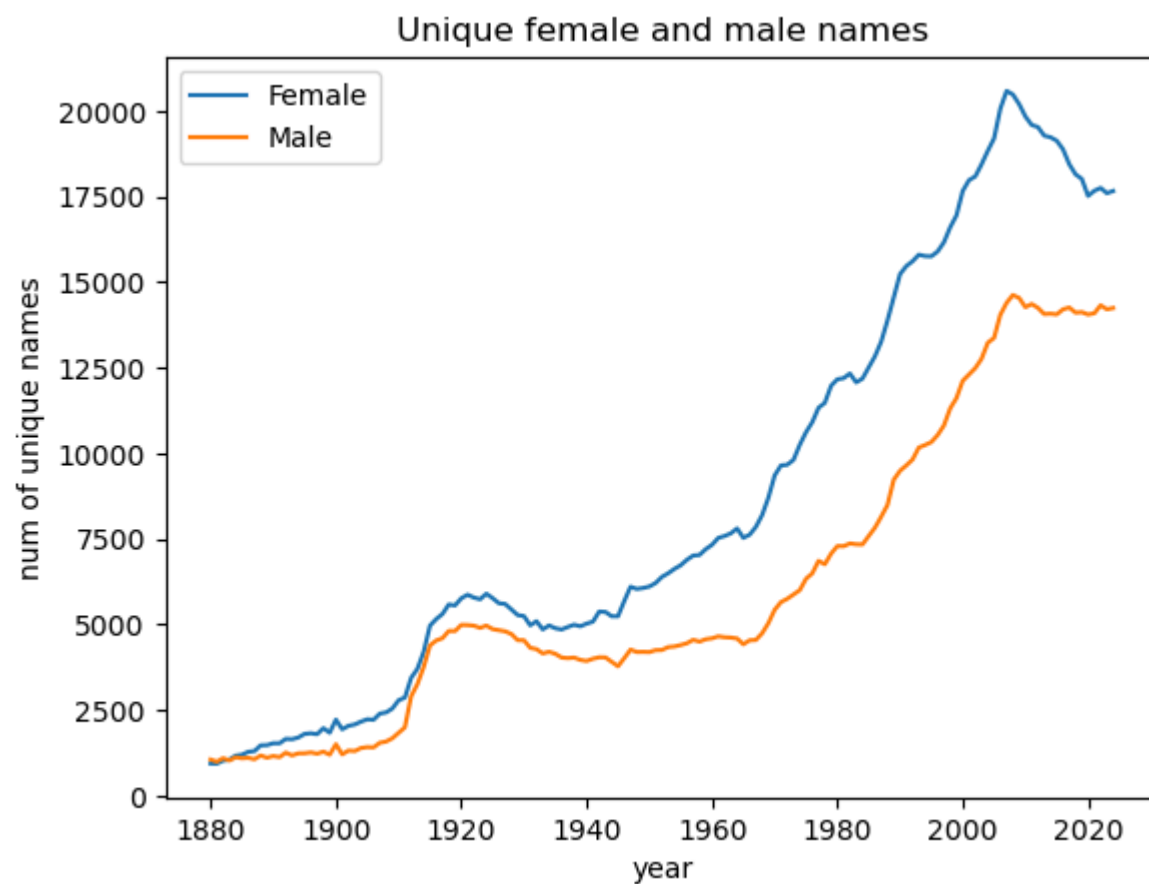
```
piv.plot()
plt.title("Unique female and male names")
plt.xlabel("year")
plt.ylabel("num of unique names")
plt.legend(["Female", "Male"])
```

Pivoted data:

	name	
gender	F	M
year		
1880	942	1058
1881	938	996
1882	1028	1099
1883	1054	1030
1884	1172	1125
...	...	...
2020	17517	14054
2021	17666	14090
2022	17750	14322
2023	17597	14206
2024	17661	14243

[145 rows x 2 columns]

Out[156... <matplotlib.legend.Legend at 0x753fb3c45ad0>



Task 5. (Most popular names over the years by gender)

In this task, we want to identify which names were the **most popular in each year**, separately for females and males.

To do this, we will build **two DataFrames**: one for female names and one for male names.

1. Split the dataset into two subsets: one containing only female names (`gender == 'F'`) and one containing only male names (`gender == 'M'`).
2. Within each subset, group the data by `year` and select the name with the **highest number of births** for that year.
3. Reset the index so the DataFrame has clean, continuous row numbers (instead of carrying over indices from the original dataset).
4. Save the results into two separate DataFrames: `most_popular_female` and `most_popular_male` .
5. Preview the results to confirm that each DataFrame contains one row per year with the most popular name and its birth count.

In [157...

```
women=names[names["gender"]=="F"]
men=names[names["gender"]=="M"]

women=women.reset_index(drop=True)
men=men.reset_index(drop=True)
most_popular_female=women.loc[women.groupby("year")["births"].idxmax()] #we can "skip" point 2., as in this way we get the same result, but
most_popular_male=men.loc[men.groupby("year")["births"].idxmax()]
print(f"Female:\n{most_popular_female}")
print(f"\nMale:\n{most_popular_male}")
```

Female:

	name	gender	births	year	last_letter
0	Mary	F	7065	1880	y
942	Mary	F	6919	1881	y
1880	Mary	F	8148	1882	y
2908	Mary	F	8012	1883	y
3962	Mary	F	9217	1884	y
...	...	...	...	...	...
1175235	Olivia	F	17682	2020	a
1192752	Olivia	F	17841	2021	a
1210418	Olivia	F	16654	2022	a
1228168	Olivia	F	15313	2023	a
1245765	Olivia	F	14718	2024	a

[145 rows x 5 columns]

Male:

	name	gender	births	year	last_letter
0	John	M	9655	1880	n
1058	John	M	8768	1881	n
2054	John	M	9557	1882	n
3153	John	M	8894	1883	n
4183	John	M	9388	1884	n
...	...	...	...	...	...
815136	Liam	M	19858	2020	m
829190	Liam	M	20407	2021	m
843280	Liam	M	20563	2022	m
857602	Liam	M	20886	2023	m
871808	Liam	M	22164	2024	m

[145 rows x 5 columns]

Task 6. (How many different male an female names were the most popular?)

In this task, we want to determine **which male and female names reached the #1 spot** (the most popular name in a given year) and **how many years each of them held that position**.

- Use the DataFrames `most_popular_male` and `most_popular_female` created in the previous task.
- Apply the function `value_counts()`.

In [158...

```
print(f'Female #1 spot names:\n{most_popular_female["name"].value_counts()}')
print(f'\nMale #1 spot names:\n{most_popular_male["name"].value_counts()}')
```

Female #1 spot names:

name

Mary 76

Jennifer 15

Emily 12

Jessica 9

Lisa 8

Linda 6

Emma 6

Olivia 6

Sophia 3

Ashley 2

Isabella 2

Name: count, dtype: int64

Male #1 spot names:

name

John 44

Michael 44

Robert 17

Jacob 14

James 13

Liam 8

Noah 4

David 1

Name: count, dtype: int64

Task 7. (Last-letter proportions in 1910, 1960, 2010 by gender)

In 2007, baby name researcher Laura Wattenberg pointed out that the distribution of boy names by final letter has changed significantly over the last 100 years.

In this task, we will analyze the **last letter of baby names** and compare proportions in the years **1910, 1960, and 2010**, separately for males (`M`) and females (`F`).

1. Create a new column `last_letter` that contains the final letter of each name (try `pandas.DataFrame.map`).
2. Filter the dataset to keep only the years 1910, 1960, and 2010.
3. Build a pivot table of total births**
 - Index: `last_letter`
 - Columns: a MultiIndex of `gender` and `year`
 - Values: sum of `births`
4. Divide each column by its column sum so each (gender, year) column sums to 1. Save as `letter_prop`.
5. Plot bar charts for `M` and `F`. Create a figure with two axes (Male on top, Female on bottom). Add titles and turn off the legend for the second plot (optional).

In [162...

```
names["last_letter"]=names["name"].map(lambda x: x[-1])
every_fifty_years=names[names["year"].isin([1910,1960,2010])]

letters_pivot=every_fifty_years.pivot_table(
    index="last_letter",
    columns=["gender", "year"],
    values="births",
    aggfunc="sum"
)

letter_prop=letters_pivot/letters_pivot.sum() #normalised
print(f"Normalised pivot table of letters frequency:\n\n{letter_prop}")
```

Normalised pivot table of letters frequency:

gender	F			M		
year	1910	1960	2010	1910	1960	2010
last_letter						
a	0.273388	0.341883	0.381228	0.005031	0.002447	0.015068
b	NaN	0.000343	0.000256	0.002116	0.001835	0.020496
c	0.000013	0.000024	0.000539	0.002482	0.007250	0.012186
d	0.017026	0.001846	0.001488	0.113852	0.122943	0.023392
e	0.336943	0.215144	0.178435	0.147599	0.083836	0.067970
f	NaN	0.000010	0.000055	0.000783	0.004331	0.001206
g	0.000144	0.000156	0.000376	0.002260	0.009487	0.001407
h	0.051531	0.036222	0.076021	0.045565	0.037912	0.051839
i	0.001526	0.039966	0.031698	0.000844	0.000609	0.022672
j	NaN	NaN	0.000093	NaN	NaN	0.000774
k	0.000121	0.000155	0.000356	0.036579	0.049366	0.018530
l	0.043188	0.033870	0.026388	0.065023	0.104886	0.070454
m	0.001200	0.008612	0.002602	0.058046	0.033828	0.024706
n	0.079238	0.130675	0.140047	0.143388	0.152521	0.362396
o	0.001672	0.002438	0.001247	0.017064	0.012833	0.042551
p	0.000018	0.000023	0.000020	0.003172	0.005675	0.001268
q	NaN	NaN	0.000030	NaN	NaN	0.000180
r	0.013395	0.006765	0.018045	0.064472	0.031049	0.087392
s	0.039036	0.012761	0.013339	0.130798	0.102694	0.065160
t	0.027432	0.015196	0.007837	0.072886	0.065645	0.022856
u	0.000683	0.000575	0.000416	0.000124	0.000057	0.001228
v	NaN	0.000060	0.000117	0.000113	0.000036	0.001450
w	0.000020	0.000031	0.001191	0.006323	0.007704	0.016182
x	0.000015	0.000037	0.000731	0.003965	0.001851	0.008597
y	0.110975	0.152549	0.116747	0.077345	0.161020	0.058210
z	0.002436	0.000658	0.000700	0.000170	0.000183	0.001830

```
In [165... fig, axes = plt.subplots(nrows=2, ncols=1, figsize=(12, 8))

#male plot
letter_prop["M"].plot(kind="bar", ax=axes[0], rot=0) #letters were vertical by default
axes[0].set_title("Frequency of male names' last letter")
axes[0].set_ylabel("Proportion")
axes[0].legend(title="Year")

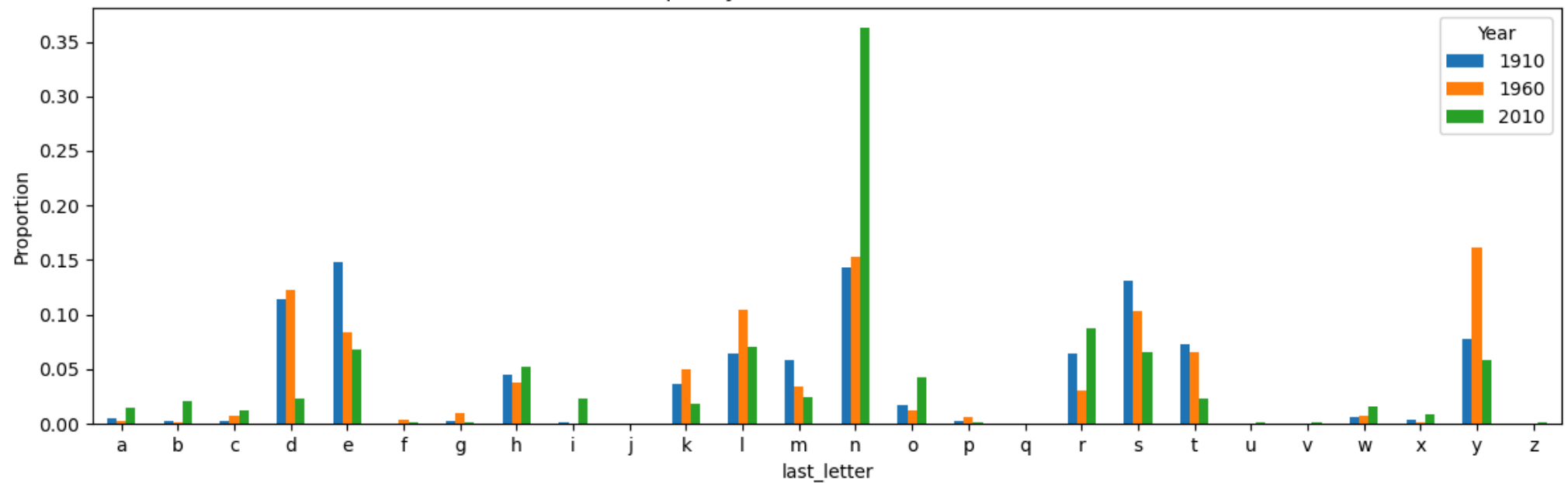
#female plot
letter_prop["F"].plot(kind="bar", ax=axes[1], rot=0)
axes[1].set_title("Frequency of female names' last letter")
axes[1].set_ylabel("Proportion")
axes[1].set_xlabel("Last letter")
axes[1].legend(title="Year")
```

```
axes[1].legend().set_visible(False)
```

```
plt.tight_layout()
```

```
plt.show()
```

Frequency of male names' last letter



Frequency of female names' last letter

